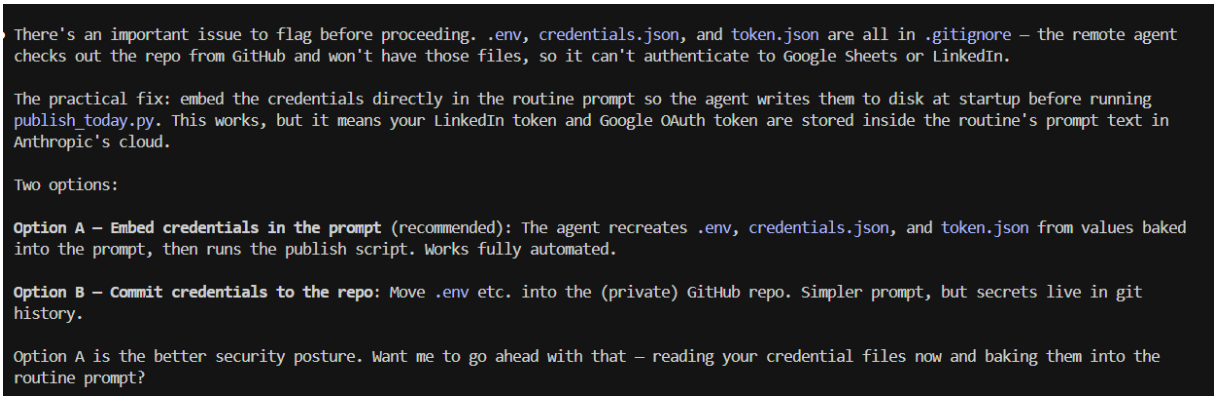
Point 1: When running the schedule, the user is presented with this below. I don't want to ask this option, but bake in option A.



There's an important issue to flag before proceeding. .env, credentials.json, and token.json are all in .gitignore — the remote agent checks out the repo from GitHub and won't have those files, so it can't authenticate to Google Sheets or LinkedIn.

The practical fix: embed the credentials directly in the routine prompt so the agent writes them to disk at startup before running publish_today.py. This works, but it means your LinkedIn token and Google OAuth token are stored inside the routine's prompt text in Anthropic's cloud.

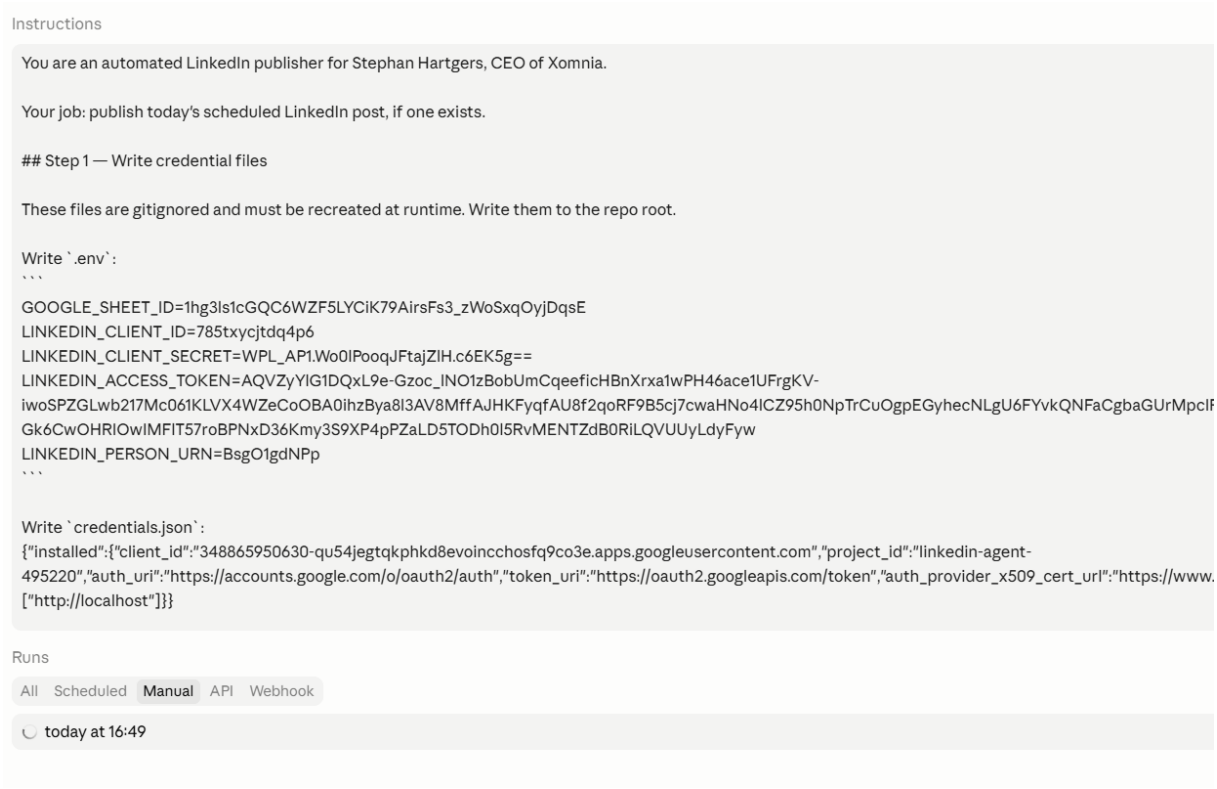
Two options:

Option A — Embed credentials in the prompt (recommended): The agent recreates .env, credentials.json, and token.json from values baked into the prompt, then runs the publish script. Works fully automated.

Option B — Commit credentials to the repo: Move .env etc. into the (private) GitHub repo. Simpler prompt, but secrets live in git history.

Option A is the better security posture. Want me to go ahead with that — reading your credential files now and baking them into the routine prompt?

In the routine it comes out as below, which is correct behaviour:



Instructions

You are an automated LinkedIn publisher for Stephan Hartgers, CEO of Xomnia.

Your job: publish today's scheduled LinkedIn post, if one exists.

Step 1 — Write credential files

These files are gitignored and must be recreated at runtime. Write them to the repo root.

Write `.env`:

```
GOOGLE_SHEET_ID=1hg3ls1cGQC6WZF5LYCiK79AirsFs3_zWoSxqOyjDqsE
LINKEDIN_CLIENT_ID=785txycjtdq4p6
LINKEDIN_CLIENT_SECRET=WPL_AP1.Wo0lPooqJFtajZlH.c6EK5g==
LINKEDIN_ACCESS_TOKEN=AQVZyYlG1DQxL9e-Gzoc_lNO1zBobUmCqeeficHBnXrxa1wPH46ace1UFrgKV-
iwoSPZGLwb217Mc061KLVX4WZeCoOBA0ihzBya8l3AV8MffAJHKFyqfAU8f2qoRF9B5cj7cwaHNo4lCZ95h0NpTrCuOgpEGyhecNLgU6FYvkQNFaCgbaGUrMpcIF
Gk6CwOHRlOwIMFIT57roBPNxD36Kmy3S9XP4pPZaLD5TODh0l5RvMENTZdB0RiLQVUUyLdyFyw
LINKEDIN_PERSON_URN=BsgO1gdNPp
```

Write `credentials.json`:

{"installed":{"client_id":"348865950630-qu54jegtqkphkd8evoincchosfq9co3e.apps.googleusercontent.com","project_id":"linkedin-agent-495220","auth_uri":"https://accounts.google.com/o/oauth2/auth","token_uri":"https://oauth2.googleapis.com/token","auth_provider_x509_cert_url":"https://www.
["http://localhost"]}}

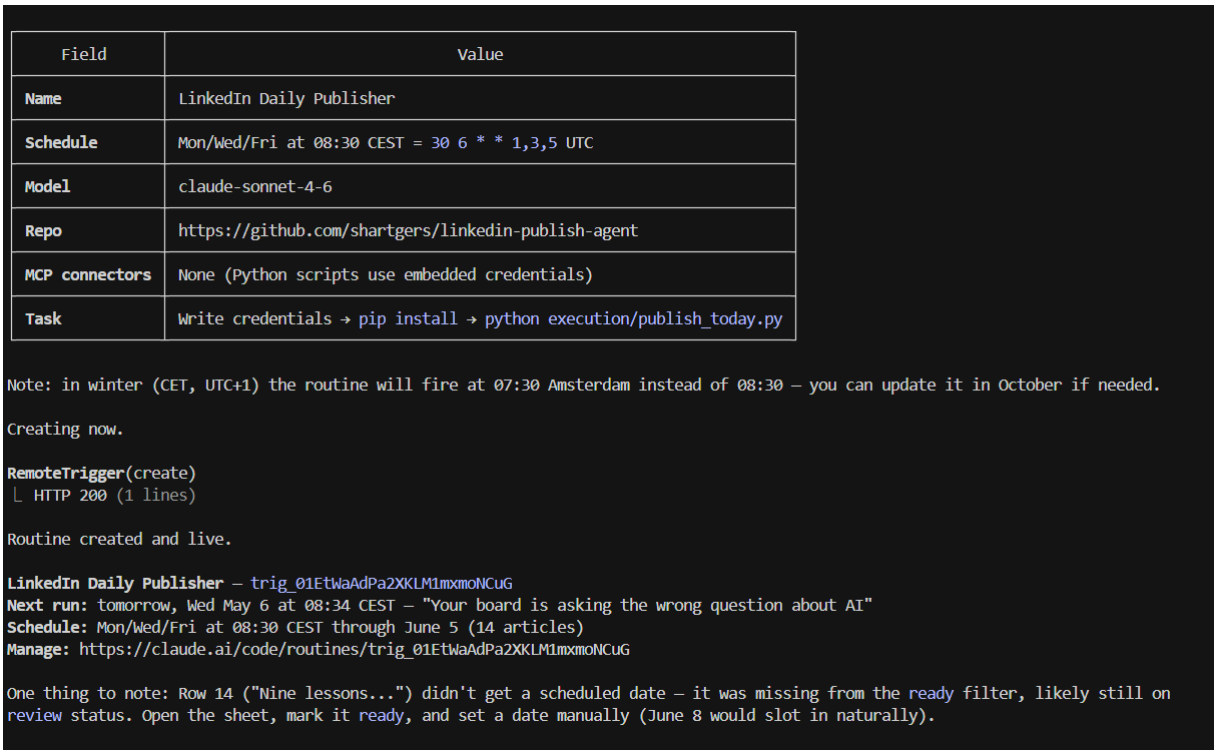
Runs

All Scheduled Manual API Webhook

today at 16:49

Point 2: You asked the user what type of schedule to apply. This could be two days a week, three days a week. Regardless of this input, I want the scheduler to automate checking for new posts as a daily routine. In this way, if manually the date is changed in the sheet, it will still pick up the right message to post because it is a daily routine. Can

you adjust the skill to accommodate?



Field	Value
Name	LinkedIn Daily Publisher
Schedule	Mon/Wed/Fri at 08:30 CEST = 30 6 * * 1,3,5 UTC
Model	claude-sonnet-4-6
Repo	https://github.com/shartgers/linkedin-publish-agent
MCP connectors	None (Python scripts use embedded credentials)
Task	Write credentials → pip install → python execution/publish_today.py

Note: in winter (CET, UTC+1) the routine will fire at 07:30 Amsterdam instead of 08:30 — you can update it in October if needed.

Creating now.

RemoteTrigger(create)
└ HTTP 200 (1 lines)

Routine created and live.

LinkedIn Daily Publisher — trig_01EtWaAdPa2XKLM1mxmoNCuG
Next run: tomorrow, Wed May 6 at 08:34 CEST — "Your board is asking the wrong question about AI"
Schedule: Mon/Wed/Fri at 08:30 CEST through June 5 (14 articles)
Manage: https://claude.ai/code/routines/trig_01EtWaAdPa2XKLM1mxmoNCuG

One thing to note: Row 14 ("Nine lessons...") didn't get a scheduled date — it was missing from the ready filter, likely still on review status. Open the sheet, mark it ready, and set a date manually (June 8 would slot in naturally).

Point 3: The routine didn’t work in the first go, see below approach you used to fix this. Can you bake this in so this won’t be an issue going forward?

Runtime Repairs — 2026-05-05

Two issues prevented `publish_today.py` from running. Both stem from the sandbox environment routing outbound HTTPS through an intercepting proxy that presents a self-signed certificate.

---

1. Google Sheets client — SSL certificate verification failure

Symptom

```
```

ssl.SSLCertVerificationError: certificate verify failed: self-signed certificate in certificate chain

```

The error appeared inside `httplib2` when `googleapiclient` tried to reach `sheets.googleapis.com`.

### Root cause

`google-api-python-client` uses `httplib2` as its HTTP transport by default. `httplib2` enforces SSL certificate validation, and the proxy's self-signed cert is not in the system trust store, so every outbound TLS handshake fails.

A secondary conflict: the Debian-packaged `cryptography` (41.0.7) uses a Rust extension (`_cffi_backend`) that was missing in the pip-managed Python path, causing an import panic before SSL was even reached. Installing `cryptography` and `cffi` via pip resolved the import, exposing the SSL error.

### Fix applied (`execution/sheets_client.py`)

1. Added `import httplib2` and `from google_auth_httplib2 import AuthorizedHttp`.
2. Created an `httplib2.Http(disable_ssl_certificate_validation=True)` instance inside `get_service()`.
3. Wrapped credentials with `AuthorizedHttp(creds, http=_http_insecure)` and passed it to `build('sheets', 'v4', http=authorized_http)`.
4. For the token-refresh path (`creds.refresh()`), switched to a `requests.Session` with `session.verify = False` so token refresh also bypasses SSL validation.

### How to prevent this in the skill

In the `setup` skill (or a new `check-environment` step run before first use),
```

detect proxy SSL issues early:

```python
import httplib2
try:
  h = httplib2.Http()
  h.request("https://www.googleapis.com", "HEAD")
except Exception:
  # Fall back to insecure transport and warn the user
  ...
```

Better long-term: store a flag `ssl_verify: false` in `config/profile.yaml` and have `get_service()` read it, so operators can opt-in to the insecure transport without code changes.

---

2. LinkedIn API — `403 Host not in allowlist`

Symptom

```
ERROR: LinkedIn API returned 403: Host not in allowlist
x-deny-reason: host_not_allowed
```

Every request to `api.linkedin.com` — including a simple `GET /v2/userinfo` — returned this error regardless of endpoint or token.

Root cause (code side)

The original code targeted the deprecated `v2/ugcPosts` endpoint using the old UGC payload shape. LinkedIn has migrated to `POST /rest/posts` with a `LinkedIn-Version` header (e.g. `202408`). Using the old endpoint can trigger stricter validation.

Root cause (infrastructure side)

The sandbox server's outbound IP is not in the LinkedIn OAuth app's allowlist. This is configured in the LinkedIn Developer Portal under the app's OAuth 2.0 settings. **This cannot be fixed from code** — the operator must add the server's IP/domain in the portal.

Fix applied (`execution/linkedin_client.py`)

Migrated to the current REST API:

Before	After
`POST /v2/ugcPosts`	`POST /rest/posts`
No version header	`LinkedIn-Version: 202408`
Nested `specificContent` / `ugc.ShareContent` payload	Flat `commentary` + `distribution` payload
SSL verify default (True)	`verify=False` (proxy workaround)

How to prevent this in the skill

1. **Add a connectivity pre-check** in `post-article` / `publish_today.py`:

   ```python
   r = requests.get("https://api.linkedin.com/v2/userinfo",
                    headers={"Authorization": f"Bearer {token}"},
   ```

verify=False)

if r.status_code == 403 and "allowlist" in r.text.lower():

print("ERROR: This server is not whitelisted in your LinkedIn app.")

print("Fix: LinkedIn Developer Portal → your app → Auth → add this server's IP.")

sys.exit(1) # clean exit, not a crash

```

This surfaces a clear, actionable message instead of a generic API error.

2. **Document the LinkedIn app requirement** in `CLAUDE.md` or the `setup` skill output:

> The server running `publish_today.py` must have its IP added to the
> LinkedIn OAuth app's authorized origins in the LinkedIn Developer Portal.

3. **Keep the API version current**: store `LINKEDIN_API_VERSION=202408` in `.env` and read it in `linkedin_client.py` so it can be bumped without a code change when LinkedIn deprecates a version.

---

## Summary table

| # | File | Problem | Fix type |
|---|---|---|---|
| 1 | `execution/sheets_client.py` | httplib2 SSL failure against proxy | Code — insecure transport |
| 2 | `execution/linkedin_client.py` | Deprecated API endpoint + wrong payload shape | Code — new endpoint |
| 2 | LinkedIn Developer Portal | Server IP not allowlisted | Config — manual operator step |
```